

Project Report: Building & Deploying HireSense AI

Vibe Coding Masterclass Series — Capstone Project Report

Project Title: HireSense AI — Intelligent Career Readiness & Interview Co-Pilot

Live Application URL: <https://hiresense-ai.vercel.app>

Backend API URL: <https://hiresense-ai-backend-ndq3.onrender.com/api/v1>

Repository: <https://github.com/Christimaria/HireSense-AI>

1. Executive Summary & Tech Stack Specifications

HireSense AI is a full-stack, AI-native career preparation platform designed to help job seekers practice interviews, calibrate resumes for ATS compatibility, and follow custom week-by-week learning roadmaps.

CODE

HIRESENSE AI ARCHITECTURE

[React 19 + Vite 8 UI]
(Vercel Deployed)

← SSE Stream →

[FastAPI Python Backend]
(Render Container)

Google GenAI SDK (v2.12.1)

▼

[Google Gemini API]
(gemini-1.5-flash)

1.1 Technical Stack Specifications

Component Layer	Technology	Primary Role & Responsibility
Frontend Framework	React 19, Vite 8, React Router v7	Responsive UI, state management, SSE chunk parsing
Styling & UI Components	Tailwind CSS v3, Lucide React	Glassmorphic dark mode, micro-animations, accessible design tokens
Backend Framework	Python 3.11, FastAPI, Pydantic v2	API routing, input validation, CORS middleware, SSE streaming
AI Integration SDK	<code>google-genai</code> (v2.12.1 SDK)	Communication with Gemini API via <code>generate_content_stream</code>
LLM Model	Google Gemini <code>gemini-1.5-flash</code>	Multimodal text & JSON generation, structured evaluation
Containerization	Docker, Docker Compose	Multistage Docker build for backend container deployment
Cloud Deployment	Vercel (Frontend), Render (Backend)	Global static hosting & cloud web service container hosting

2. Prompting Strategy & Prompt Engineering Documentation

HireSense AI implements **Role-Task-Constraint (RTC)** and **Structured JSON Schema Output** prompting patterns across 5 core AI features.

Feature 1: Mock Interview Question Generator (`interview_prompts.py`)

System Prompt (`INTERVIEW_SYSTEM_PROMPT`)

PYTHON

```
INTERVIEW_SYSTEM_PROMPT = """You are a professional technical interviewer at a top technology company.  
Your job is to conduct realistic, challenging, and fair mock interviews.
```

```
RULES:
```

- Ask ONE question at a time. Never ask multiple questions in a single response.
- Questions must be relevant to the role, experience level, and interview type.
- Do NOT repeat questions that have already been asked in this session.
- Progress naturally: start easier, build complexity as the session continues.
- For behavioral questions, set up scenarios clearly.
- For technical questions, be specific – avoid vague or generic questions.
- Return ONLY the question text. No greetings, no prefixes like "Question 3:", no explanations.

```
"""
```

User Prompt Template Builder (`build_question_prompt`)

PYTHON

```
def build_question_prompt(
    role: str,
    experience_level: str,
    interview_type: str,
    question_number: int,
    total_questions: int,
    conversation_history: list[dict],
) -> str:
    role_ctx = ROLE_CONTEXT.get(role, role)
    type_ctx = INTERVIEW_TYPE_CONTEXT.get(interview_type, "")
    exp_ctx = EXPERIENCE_CONTEXT.get(experience_level, experience_level)

    history_text = ""
    if conversation_history:
        history_lines = []
        for turn in conversation_history:
            history_lines.append(f"Q: {turn['question']}")
            history_lines.append(f"A: {turn['answer'][:300]}...")
        history_text = "\n".join(history_lines)

    return f"""
Interview Configuration:

• Role: {role}

• Experience Level: {experience_level} ({exp_ctx})

• Interview Type: {interview_type}

• Question: {question_number} of {total_questions}

• Key Topics: {role_ctx}

• Style Guidance: {type_ctx}

{"Previous conversation history:" if history_text else "This is the FIRST question of the session."}
{history_text}

Now generate question #{question_number}. Return ONLY the question text.
"""
```

Feature 2: Candidate Answer Evaluation & Grading (`evaluation_prompts.py`)

System Prompt (`EVALUATION_SYSTEM_PROMPT`)

PYTHON

```
EVALUATION_SYSTEM_PROMPT = """\
You are an expert technical recruiter, senior engineering manager, and career coach with 15+ years of experience.
Your task is to review a candidate's answer to an interview question, grade it constructively and strictly, and provide feedback.

RULES:

• Be realistic and fair. Do not inflate scores.

• High scores (8.5+) require technical accuracy, clarity, and context-appropriate detail.

• Low scores (<5.0) are appropriate for major technical gaps, misunderstandings, or excessive vagueness.

• Grading must scale according to the candidate's Experience Level.

• Return ONLY a single JSON object matching the requested schema.

"""
```

User Prompt Template Builder (`build_evaluation_prompt`)

PYTHON

```
def build_evaluation_prompt(question, answer, role, experience_level, interview_type) -> str:
    return f"""
Interview Context:

• Role: {role} | Experience Level: {experience_level} | Interview Type: {interview_type}

Question Asked:
"{question}"

Candidate's Answer:
"{answer}"

Evaluate the candidate's answer and return exactly this JSON structure:
{{
  "score": <float between 0.0 and 10.0 representing the quality of the answer>,
  "strengths": ["<strength 1>", "<strength 2>", "<strength 3>"],
  "weaknesses": ["<weakness 1>", "<weakness 2>", "<weakness 3>"],
  "improved_answer": "<a model answer tailored to the role and experience level>",
  "tips": ["<tip 1>", "<tip 2>", "<tip 3>"]
}}

Return ONLY the JSON object. No markdown fences, no extra text.

"""
```

Feature 3: Resume ATS Scanner & Reviewer ([resume_prompts.py](#))

System Prompt ([RESUME_SYSTEM_PROMPT](#))

PYTHON

```
RESUME_SYSTEM_PROMPT = """\
You are an expert technical recruiter and career coach with 15+ years of experience reviewing resumes for top tech companies.
Your task is to analyse resumes and return structured, honest, actionable feedback.

RULES:

• Be candid but constructive. Do not inflate scores.

• Scores use a 0-10 float scale.

• ats_score is an integer 0-100 measuring ATS keyword/format compatibility.

• Keep per-section feedback concise (2-3 sentences).

• Return ONLY a single JSON object.

"""
```

User Prompt Template Builder ([build_resume_review_prompt](#))

PYTHON

```
def build_resume_review_prompt(resume_text: str, target_role: str | None) -> str:
    return f"""\
Target Role: {target_role or "General Merit"}

Analyse the resume below and return exactly this JSON structure:

{{
  "overall_score": <float 0-10>,
  "ats_score": <int 0-100>,
  "sections": {{
    "summary":    {{ "score": <float>, "feedback": "<string>" }},
    "experience": {{ "score": <float>, "feedback": "<string>" }},
    "skills":     {{ "score": <float>, "feedback": "<string>" }},
    "education":  {{ "score": <float>, "feedback": "<string>" }}
  }},
  "strengths":    [<string>, ...],
  "weaknesses":   [<string>, ...],
  "recommendations": [<string>, ...]
}}

RESUME TEXT:
{resume_text}

Return ONLY the JSON object.

"""
```

Feature 4: Career Learning Roadmap Generator (`roadmap_prompts.py`)

System Prompt (`ROADMAP_SYSTEM_PROMPT`)

PYTHON

```
ROADMAP_SYSTEM_PROMPT = """\nYou are an expert career coach and senior technical trainer.\nAnalyze a candidate's current skills and target role, identify core skill gaps, and generate a customized week-by-week learning roadmap.\n\nRULES:\n\n• Be highly practical. Focus on project-based learning and hands-on milestones.\n\n• Ensure the roadmap fits the requested timeline.\n\n• Recommend reputable resources (official docs, courses, books).\n\n• Return ONLY a single JSON object matching the requested structure.\n\n"""
```

User Prompt Template Builder (`build_roadmap_prompt`)

PYTHON

```
def build_roadmap_prompt(current_skills: list[str], target_role: str, timeline: str) -> str:\n    skills_list = ", ".join(current_skills)\n    return f"""\nCandidate Context:\n\n• Current Skills: {skills_list}\n\n• Target Role: {target_role}\n\n• Timeline: {timeline}\n\nGenerate a detailed, week-by-week roadmap to bridge these skill gaps within the {timeline} timeframe.\nReturn ONLY the JSON object.\n\n"""
```

Feature 5: Holistic Performance Dashboard (`dashboard_prompts.py`)

System Prompt (`DASHBOARD_SYSTEM_PROMPT`)

```
PYTHON

DASHBOARD_SYSTEM_PROMPT = """\
You are an elite executive recruiter and technical interviewer with 15+ years of experience conducting performance evaluations.
Your task is to analyze an entire mock interview transcript and produce a detailed, honest, structured performance evaluation.

RULES:

• Evaluate the candidate holistically across Technical Depth, Communication Clarity, Problem Solving, and Contextual Awareness.
• Provide a grade (e.g. A, B+, B, C-, F) matching typical industry standards.
• Return ONLY a single JSON object matching the requested structure.

"""
```

3. Phase-by-Phase Development Summary (Vibe Coding Methodology)

Phase 1: Architecture & API Schema Definition

- Specified 5 core feature endpoints (`/interview/question` , `/evaluation/answer` , `/evaluation/dashboard` , `/resume/review` , `/roadmap/generate`).
- Created Pydantic models for request validation and response typing.

Phase 2: Backend Core & Gemini Integration

- Implemented FastAPI application factory in `app/main.py` .
- Configured `google-genai` SDK with async streaming generators in `app/ai/client.py` .
- Built custom Server-Sent Events (SSE) helpers (`stream_text_chunks` and `stream_json_object`) in `app/utils/sse.py` .

Phase 3: Frontend Interface & SSE Stream Reader

- Designed responsive glassmorphic UI using React 19 and Tailwind CSS.
- Developed `streamPost` helper utilizing browser `fetch` and `TextDecoder` streams to render progressive typing animations.

Phase 4: Containerization

- Authored multi-stage `Dockerfile` with Python 3.11 slim base image.
- Configured dependency caching and Uvicorn production entrypoint.

Phase 5: Deployment & Verification Audit

- Deployed FastAPI backend container to Render.
 - Deployed React static bundle to Vercel.
 - Resolved CORS, API URL normalization, and Gemini model fallback routing.
-

4. Technical Challenges & Resolutions

Challenge 1: Gemini Model Availability & Fallback Routing

- **Issue:** Requesting restricted or deprecated model names returned `404 NOT_FOUND: model is no longer available`.
- **Resolution:** Configured application default to `gemini-1.5-flash` and added automatic fallback handling in `client.py` to seamlessly retry with `gemini-1.5-flash` if any model returns 404.

Challenge 2: Vite Build-Time Environment Variable Baking

- **Issue:** Updating `VITE_API_URL` on Vercel did not take effect immediately due to build-time bundle compilation.
- **Resolution:** Implemented `getNormalizedApiBaseUrl()` in `api.js` to handle trailing slashes and missing `/api/v1` suffixes, followed by a cache-cleared Vercel redeployment.

Challenge 3: JSON Decoding Corruption in SSE Streams

- **Issue:** Python `str.strip("`json")` character-stripped valid trailing letters (`j`, `s`, `o`, `n`) from JSON strings, causing parse errors.
 - **Resolution:** Refactored `sse.py` with `_clean_json_text()` using index-based newline slicing for markdown code fences.
-

5. Key Learnings & Reflection

1. **Vibe Coding Acceleration:** Pairing with AI tools enabled full-stack development, prompt engineering, and cloud deployment in record time. 2. **Streaming User Experience:** Progressive SSE token rendering provides superior user engagement compared to slow, blocking HTTP requests. 3. **Decoupled Architecture Benefits:** Separating React on Vercel from FastAPI on Render ensured complete API key protection while keeping the system modular and scalable.